Seminario de Arquitecturas de Software: Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Ingeniería de Software

Equipo: HAVING FE

Integrante	**Pregunta(s)**
Eric Reyes Milián	Pregunta 1
Adrián Estévez Álvarez	Pregunta 2
Alfonso Tejeda Rodríguez	Pregunta 3
Kamila Reinoso Asín	Pregunta 4
Miguel Andrés Cazorla Zamora	Pregunta 5
Reynaldo Guillermo Capdevila Reyes	Pregunta 6

Repositorio: https://github.com/alfonsotejeda/astarship

Índice

Enunciados de las preguntas **2**

1. Pregunta 1: Análisis de trade-offs **5**
1.1. El error fundamental: el negocio depende de un detalle 5
1.2. Cómo lo resuelve la Regla de Dependencia 6
1.3. La misma idea vista como Puertos y Adaptadores 7
1.4. Impacto en la testabilidad 7
1.5. Impacto al cambiar de SQL Server a MongoDB 8
1.6. Conclusión 9

2. Pregunta 2: La simetría del hexágono vs. la jerarquía de las capas **9**

3. Pregunta 3: Aplicación de DDD y Bounded Context en un dominio complejo **9**

4. Pregunta 4: Evaluación de la Inversión de Dependencias en la práctica **13**
4.1. Marco normativo: DIP y Regla de Dependencia 14
4.2. Inciso a): Violación de los principios arquitectónicos 14
4.3. Inciso b): Rediseño arquitectónico 17
4.4. Inciso c): Pruebas aisladas y mantenibilidad 20

5. Pregunta 5: El Kitty Problem y el Principio Abierto/Cerrado **21**
5.1. Descripción del problema 21
5.2. ¿Por qué la división por verbos provocó el problema? 22
5.3. Diseño basado en DDD y Onion/Clean 22
5.4. Principio de Abierto/Cerrado y papel de los Adaptadores 22

6. Pregunta 6: N-Capas frente a requisitos de atributos de calidad **23**

Enunciados de las preguntas

1. **Análisis de Trade-offs (Arquitectura N-Capas vs. Clean/Onion/Hexagonal):** Un equipo de desarrollo está construyendo un sistema de gestión de inventario. Inicialmente, eligen una Arquitectura N-Capas tradicional por su simplicidad y familiaridad. Después de un año, el sistema se ha vuelto un monolito difícil de mantener, donde los cambios en la base de datos afectan la lógica de negocio y la interfaz de usuario, y las pruebas unitarias son lentas y frágiles.

   Con base en los principios de la Arquitectura Limpia y la Arquitectura Hexagonal, explique detalladamente qué error fundamental de acoplamiento se está cometiendo y cómo la introducción de la Regla de Dependencia (Martin) o el uso de Puertos y Adaptadores (Cockburn) resolverían estructuralmente este problema. ¿Cuál sería el impacto en la testabilidad y la capacidad de cambiar la base de datos de SQL Server a MongoDB sin afectar la lógica de negocio?

2. **Comparación de Estilos: La Simetría del Hexágono vs. la Jerarquía de las Capas:** La Arquitectura Hexagonal de Cockburn y la Arquitectura Onion de Palermo comparten el objetivo de aislar el núcleo de negocio, pero utilizan representaciones visuales y metáforas radicalmente diferentes (un hexágono simétrico frente a capas concéntricas).

   Analice en profundidad qué problema conceptual resuelve la simetría de la representación hexagonal que una representación de capas apiladas (como la de Onion o la N-Capas tradicional) no logra transmitir de forma intuitiva. ¿Cómo se traduce el concepto de Puerto y Adaptador en la práctica de definir interfaces en las capas internas de la Arquitectura Onion? ¿Por qué Cockburn afirma que el hexágono no tiene seis lados por una razón numérica, sino para eliminar la asimetría izquierda-derecha o arriba-abajo?

3. **Aplicación de DDD y Bounded Context en un Dominio Complejo:** Una empresa de seguros necesita desarrollar un sistema para gestionar pólizas de automóviles. El sistema debe interactuar con un departamento de siniestros, un sistema de facturación de primas y una aplicación para que los ajustadores de seguros evalúen daños en el lugar del accidente. Cada uno de estos subsistemas tiene su propia terminología y preocupaciones (ej: póliza para un actuario es un contrato con cláusulas, para facturación es una fuente de ingresos, y para un ajustador es un número de referencia).

   Basado en los conceptos de Domain-Driven Design (Evans) y Bounded Context (Richards & Ford), explique cómo dividiría este sistema en varios contextos delimitados. ¿Cómo evitaría que el modelo de Póliza se contamine con detalles de siniestros o facturación? ¿Cómo se comunicarían estos contextos de manera desacoplada? Relacione esta solución con la implementación de una Arquitectura Orientada al Dominio (como Onion o Clean) y la creación de un Adaptador de Servicio para la comunicación externa, tal como se describe en la Guía de Arquitectura N-Capas.

4. **Evaluación de la Inversión de Dependencias en la Práctica:** Un equipo de desarrollo está trabajando en un sistema de gestión de pedidos. La lógica de negocio principal se encuentra en una clase `OrderProcessor` que, para completar su tarea, necesita comunicarse con un servicio externo de validación de direcciones. La implementación actual tiene una dependencia directa: `OrderProcessor` instancia y llama directamente

a la clase `AddressValidatorService`, que a su vez depende de una librería HTTP específica para comunicarse con el servicio externo.

Con base en los principios de la Inversión de Dependencias (Martin) y la Regla de Dependencia (Martin), analice y responda:

a) ¿Por qué esta dependencia directa viola los principios fundamentales de las arquitecturas Hexagonal, Limpia y Onion? Explique específicamente qué dirección deberían tener las dependencias del código fuente y por qué la implementación actual no la cumple.

b) Rediseñe la interacción entre `OrderProcessor` y `AddressValidatorService` para que se alinee con los principios de estas arquitecturas. Defina claramente qué componente (interfaz) debe residir en el núcleo de la aplicación y qué componente debe residir en la capa de infraestructura o en el exterior del hexágono. Justifique su diseño basándose en el principio de que los detalles deben depender de las abstracciones.

c) Explique cómo este nuevo diseño permite que la lógica de negocio (`OrderProcessor`) sea probada de forma aislada, sin necesidad de realizar llamadas HTTP reales al servicio externo, y sin depender de la librería HTTP subyacente. ¿Qué implicaciones tiene este diseño para la mantenibilidad y la capacidad de cambiar el proveedor de validación de direcciones en el futuro?

5. **Análisis de un Problema Real: El Kitty Problem y el Principio de Abierto/Cerrado:** En su libro *Fundamentals of Software Architecture*, Richards y Ford presentan el *Kitty Problem* para ilustrar un fallo común en el diseño de arquitecturas orientadas a servicios. Describa este problema: un requisito transversal (como la entrega de gatitos) afecta a múltiples servicios previamente existentes (TaxiUI, TaxiFinder, TaxiSelector, etc.), requiriendo cambios coordinados en todos ellos.

   ¿Por qué la solución de dividir el sistema en servicios basados en verbos o entidades funcionales (como TaxiFinder o TaxiSelector) llevó a este problema? Basándose en los principios de DDD y la Arquitectura Onion/Clean, ¿cómo se podría haber diseñado el sistema para que un cambio de este tipo afectara a un solo componente o servicio? Proponga una solución utilizando componentes como Kittens y Rides que se añadan como plugins, explicando cómo esto respeta el Principio de Abierto/Cerrado (OCP) y qué papel juegan los Adaptadores en esta extensión.

6. **Evaluación de la Arquitectura N-Capas frente a Requisitos de Atributos de Calidad:** La Arquitectura N-Capas es criticada por su bajo rendimiento en características como Agilidad general, Facilidad de despliegue y Escalabilidad (Richards, 2015). Un estudiante propone que, con el uso de capas abiertas (*open layers*) y técnicas como el *fast-lane reader*, se pueden mitigar estos problemas.

   Analice esta afirmación. ¿Cómo afecta la introducción de capas abiertas al principio de Aislamiento de Capas y a la mantenibilidad a largo plazo? ¿Por qué la solución de capas abiertas puede llevar a un *sinkhole anti-pattern* y qué dice la regla del 80/20 sobre esto? Finalmente, explique por qué, para un sistema con requisitos de alta escalabilidad y agilidad, una arquitectura como la Hexagonal o la Limpia, que se basan en los principios

de DDD y desacoplamiento, son inherentemente más adecuadas que una Arquitectura N-Capas, incluso con optimizaciones.

1. Pregunta 1: Análisis de trade-offs (N-Capas vs. Clean/Onion/Hexagonal)

El caso del sistema de inventario es común: la arquitectura N-Capas tradicional puede funcionar bien al principio y, con el tiempo, el sistema volverse difícil de modificar. Los síntomas descritos —cambios en la base de datos que se propagan hasta la lógica de negocio y la interfaz, junto con pruebas unitarias lentas y frágiles— pueden tener una causa común: un acoplamiento excesivo entre las reglas del negocio y los detalles de infraestructura. A continuación, analizamos por qué ocurre y cómo puede corregirse.

1.1. El error fundamental: el negocio depende de un detalle

El problema no está en utilizar capas, sino en la dirección de sus dependencias. En una arquitectura N-Capas tradicional, la Presentación depende de la capa de Negocio y esta, a su vez, depende de la capa de Acceso a Datos, que interactúa con la base de datos (Richards, 2015). Cuando la lógica de negocio utiliza directamente componentes, modelos o contratos específicos de la persistencia, la parte más estable y valiosa del sistema queda acoplada a un detalle tecnológico que puede cambiar con mayor frecuencia.

Esto entra en conflicto con el Principio de Inversión de Dependencias: los módulos de alto nivel no deberían depender directamente de los módulos de bajo nivel; ambos deberían depender de abstracciones (Martin, 2017). En el sistema de inventario, este acoplamiento puede producir tres consecuencias:

- **Propagación de cambios.** Si las entidades del ORM, las consultas o los contratos de persistencia se utilizan directamente desde la lógica de negocio o la presentación, un cambio en el esquema de la base de datos puede obligar a modificar varias capas. Esto no ocurre necesariamente en toda arquitectura N-Capas, sino cuando los límites entre ellas no aíslan adecuadamente los detalles de persistencia.

- **Un dominio moldeado por la tecnología.** Las entidades del negocio pueden terminar reflejando la estructura de las tablas o las clases generadas por el ORM, en lugar de representar los conceptos y reglas propios del inventario. De este modo, las decisiones de persistencia influyen innecesariamente en el modelo de dominio.

- **Pruebas acopladas a la infraestructura.** Cuando un caso de uso depende directamente de una implementación concreta de acceso a datos, probar una regla de negocio puede exigir una base de datos real o simulaciones complejas. Las pruebas pueden entonces depender de la red, de datos precargados y del estado del esquema, volviéndose más lentas y frágiles. Esto no es inevitable en N-Capas, pero sí un riesgo cuando no se separan las abstracciones de sus implementaciones.

Conviene aclarar que la descripción anterior se refiere a una implementación tradicional en la que la lógica de negocio depende de detalles de persistencia. La guía de Microsoft sobre arquitectura N-Capas orientada al dominio muestra una variante que sitúa las interfaces de los repositorios en una capa interna, lo que permite desacoplar el dominio de las implementaciones concretas de acceso a datos (De la Torre Llorente et al., 2010). Por tanto, no todas las arquitecturas por capas presentan necesariamente el mismo problema: el resultado depende de cómo se definan y respeten sus límites.

1.2. Cómo lo resuelve la Regla de Dependencia

La Arquitectura Limpia propone que las dependencias del código fuente apunten hacia adentro, en dirección a las políticas de más alto nivel (Martin, 2017). En su representación habitual, las entidades se sitúan en el centro; después aparecen los casos de uso; y, hacia el exterior, los adaptadores de interfaz y los frameworks y drivers, donde suelen encontrarse la interfaz de usuario, la base de datos y otras tecnologías. Los componentes internos no deben depender de clases, funciones o formatos definidos exclusivamente en los círculos externos.

En el sistema de inventario, un caso de uso como `RegistrarEntradaDeStock` necesita recuperar y guardar un producto, pero no debería conocer SQL Server ni MongoDB. Para lograrlo, el núcleo define la interfaz que necesita y la infraestructura proporciona una implementación concreta. En tiempo de ejecución, el caso de uso puede invocar esa implementación a través de la interfaz; sin embargo, la dependencia del código fuente apunta desde la implementación concreta hacia la abstracción definida en el núcleo.

```
// Nucleo de la aplicacion: no depende de un motor de base de datos.
public interface IProductoRepository
{
    Producto ObtenerPorSku(string sku);
    void Guardar(Producto producto);
}

// Caso de uso: depende de la abstraccion del repositorio.
public class RegistrarEntradaDeStock
{
    private readonly IProductoRepository _repo;

    public RegistrarEntradaDeStock(IProductoRepository repo)
    {
        _repo = repo;
    }

    public void Ejecutar(string sku, int cantidad)
    {
        var producto = _repo.ObtenerPorSku(sku);
        producto.AgregarStock(cantidad); // Regla de negocio
        _repo.Guardar(producto);
    }
}

// Infraestructura: implementa el puerto definido por el nucleo.
public class SqlServerProductoRepository : IProductoRepository
{
    public Producto ObtenerPorSku(string sku)
    {
        // Implementacion concreta mediante EF Core o ADO.NET.
        throw new NotImplementedException();
    }

    public void Guardar(Producto producto)
    {
        // Implementacion concreta mediante EF Core o ADO.NET.
        throw new NotImplementedException();
    }
```

```
40 }
```

Código 1: Inversión de dependencias en el sistema de inventario

El fragmento es ilustrativo: presupone que la clase `Producto` y sus reglas están definidas en el núcleo, y que los métodos del repositorio se implementan utilizando la tecnología seleccionada. En una aplicación real también habría que definir cómo se gestionan los casos en que el producto no existe o la cantidad recibida no es válida.

La implementación concreta se proporciona al caso de uso desde la raíz de composición, es decir, el punto de arranque donde se registran y conectan las dependencias. Ese es el lugar que conoce tanto la abstracción del núcleo como la implementación concreta de infraestructura.

1.3. La misma idea vista como Puertos y Adaptadores

Cockburn (2005) plantea una estructura equivalente a partir de otro objetivo: que la aplicación pueda utilizarse desde distintos entornos —por una persona, otro programa, una prueba automatizada o un script— y que pueda desarrollarse y probarse sin depender de dispositivos o bases de datos reales. Para ello distingue dos conceptos:

- El **puerto** es una interfaz que representa una operación o necesidad de la aplicación, como guardar un producto o registrar una entrada de stock. Debe expresar una intención del negocio o de la aplicación, no una tecnología concreta.
- El **adaptador** es un componente externo que conecta el puerto con una tecnología o mecanismo específico, como SQL Server, MongoDB, una API REST o una implementación de prueba.

En el ejemplo, `IProductoRepository` funciona como un puerto de salida y `SqlServerProductoRepository` como su adaptador concreto. Un controlador HTTP, por su parte, puede actuar como adaptador de entrada al traducir una petición externa en una llamada a un caso de uso.

La Arquitectura Onion de Palermo (2008) sigue una idea relacionada mediante capas concéntricas: el modelo de dominio ocupa el centro, las abstracciones necesarias se definen en capas internas y la infraestructura las implementa desde el exterior. Aunque las metáforas y la organización visual difieren, Clean, Onion y Hexagonal buscan proteger el núcleo frente a los detalles externos y permitir que estos se sustituyan sin modificar innecesariamente las reglas de negocio.

1.4. Impacto en la testabilidad

Cuando los casos de uso dependen de abstracciones, las pruebas del núcleo no necesitan utilizar obligatoriamente la base de datos real. Se puede crear un adaptador de prueba en memoria, por ejemplo, una clase `ProductoRepositoryEnMemoria` respaldada por un diccionario. Esto aporta varias ventajas:

- Las pruebas de reglas de negocio y de casos de uso pueden ejecutarse sin acceder a la red ni al disco, por lo que normalmente son más rápidas.

- Las pruebas pueden ser más deterministas, ya que se reduce su dependencia del estado de una base de datos, de la configuración de la infraestructura y de los datos compartidos. Aun así, el determinismo también depende de que las propias pruebas estén correctamente diseñadas.

- La integración con la base de datos real se verifica mediante un conjunto específico de pruebas de integración o de contrato, que comprueba que el adaptador implementa correctamente el puerto definido por la aplicación.

Así se favorece una pirámide de pruebas equilibrada: muchas pruebas rápidas para la lógica del núcleo y un conjunto más reducido de pruebas de integración para verificar los adaptadores y la infraestructura. Estas últimas siguen siendo necesarias, porque las pruebas unitarias con repositorios en memoria no demuestran por sí solas que las consultas, transacciones o configuraciones de la base de datos real funcionen correctamente.

1.5. Impacto al cambiar de SQL Server a MongoDB

Con esta estructura, desde la perspectiva del código de aplicación, la sustitución del motor se concentra principalmente en dos pasos: implementar un adaptador `MongoProductoRepository` que satisfaga el contrato de `IProductoRepository` y actualizar el registro de dependencias en la raíz de composición. Las entidades del dominio, los casos de uso y sus pruebas unitarias pueden permanecer sin cambios, siempre que la nueva implementación respete el contrato y que dicho contrato no exponga particularidades de SQL Server.

No obstante, la migración completa no es gratuita ni se limita a cambiar una clase. Hay que reescribir y probar el adaptador, migrar los datos existentes y tratar las diferencias entre ambos sistemas, como las garantías transaccionales, la representación de relaciones, los índices y las formas de consulta. La inversión de dependencias acota el impacto en el código de aplicación, pero no elimina el trabajo de migración ni las decisiones de diseño asociadas.

Para que el desacoplamiento sea efectivo, conviene tener en cuenta tres aspectos:

- **El puerto debe hablar el lenguaje del dominio.** Si la interfaz expone detalles específicos de una tecnología, como `IQueryable`, consultas SQL o identificadores autoincrementales, esos detalles pueden filtrarse hacia el núcleo y reducir la independencia que se pretendía conseguir.

- **El costo se concentra, pero no desaparece.** Es necesario desarrollar y probar el nuevo adaptador, adaptar la persistencia y verificar que se conservan las reglas y garantías que necesita la aplicación.

- **Los agregados de DDD orientan, pero no determinan, el almacenamiento.** Las fronteras de los agregados ayudan a identificar qué información debe mantener su consistencia conjuntamente. Sin embargo, no implican que cada agregado deba almacenarse como un único documento en MongoDB. Por ejemplo, en el inventario puede ser conveniente guardar el producto y su stock actual en un documento y almacenar los movimientos históricos por separado, según el volumen de datos, los patrones de consulta y los requisitos de consistencia.

1.6. Conclusión

El error del equipo no fue elegir una arquitectura por capas, sino permitir que el núcleo del negocio dependiera directamente de detalles de persistencia. Invertir esa dependencia, mediante la Regla de Dependencia de Martin o mediante los Puertos y Adaptadores de Cockburn, permite que las reglas de negocio permanezcan aisladas y que la base de datos se trate como un detalle reemplazable.

Esta decisión tiene un costo: puede introducir más interfaces, indirección y disciplina de diseño desde las primeras etapas. En una aplicación CRUD pequeña y sencilla, ese nivel de abstracción podría no compensar. Sin embargo, en un sistema de inventario que ya ha demostrado ser difícil de mantener, separar el núcleo de la infraestructura puede mejorar la mantenibilidad, acelerar las pruebas de la lógica de negocio y reducir el alcance de los cambios tecnológicos. El beneficio real depende de que las interfaces representen necesidades de la aplicación y de que las capas respeten las direcciones de dependencia establecidas.

2. Pregunta 2: La simetría del hexágono vs. la jerarquía de las capas

El problema de la representación de la arquitectura por capas es que solo presenta dos lados de distinta naturaleza, la UI arriba y la base de datos abajo, y nada obliga a cumplir las reglas, por lo que puede terminar habiendo lógica filtrada entre capas (Cockburn, 2005). Mientras tanto, en la arquitectura hexagonal no hay un "arriba" o "abajo", sino un adentro y un afuera (Cockburn, 2005). Toda la lógica está dentro, en el núcleo, mientras que todo lo externo, como la UI, la base de datos y los tests, está afuera y se maneja por igual mediante puertos y adaptadores (Cockburn, 2005).

En la arquitectura Onion, la interfaz definida en el núcleo hace el papel de puerto, y el adaptador sería la clase de la capa externa que la implementa. No usan los mismos nombres, pero cumplen con la idea de externalizar la infraestructura mediante adaptadores. La interfaz está en el núcleo y este es estable, ya que se expresa en términos del dominio. La implementación depende de tecnologías concretas y puede cambiar sin afectar nada. Así se logra la inversión de dependencias, para que el núcleo no dependa de nada externo (Martin, 2017; Palermo, 2008).

El hexágono no tiene seis lados por razón numérica, sino para dar espacio a insertar puertos y adaptadores sin quedar atrapado en capas (Cockburn, 2005). Tampoco elimina la asimetría izquierda-derecha: la deja de lado un momento para destacar la asimetría de "adentro" y "afuera", y trata todo lo externo de la misma manera. Después la retoma al distinguir los actores primarios, que conducen la aplicación, de los secundarios, que ella conduce (Cockburn, 2005).

3. Pregunta 3: Aplicación de DDD y Bounded Context en un dominio complejo

El síntoma: un término, cuatro modelos

El escenario planteado describe un sistema en el que el término *póliza* designa cuatro realidades distintas según quién lo pronuncie: para el actuario es un contrato con cláusulas

y coberturas; para el departamento de siniestros, el expediente contra el que se presenta una reclamación; para facturación, una fuente de ingresos recurrentes; y para el ajustador que evalúa los daños en el lugar del accidente, apenas un número de referencia.

Conviene empezar por reconocer de qué clase de problema se trata. No es una imprecisión terminológica que pueda resolverse acordando una definición común entre departamentos, sino una propiedad del dominio: cada área necesita de la póliza precisamente aquello que le permite ejecutar su trabajo, y las reglas de negocio que debe hacer cumplir sobre ella difieren entre sí. Evans (2003) sostiene que el intento de construir un modelo único que unifique un dominio extenso conduce a uno de dos resultados, ambos indeseables: un modelo tan genérico que deja de expresar las reglas del negocio, o un modelo que acumula las preocupaciones de todas las áreas y que, en consecuencia, nadie puede modificar sin afectar a las demás. La alternativa que propone no consiste en unificar mejor, sino en renunciar deliberadamente a la unificación total y delimitar de forma explícita el ámbito de validez de cada modelo.

División en contextos delimitados

Un *Bounded Context* o contexto delimitado es la frontera explícita dentro de la cual un modelo de dominio concreto está definido y resulta aplicable, y dentro de la cual su terminología —el *Lenguaje Ubicuo*— es consistente y carece de ambigüedad (Evans, 2003). La frontera es simultáneamente lingüística, organizativa y técnica: delimita un vocabulario, el equipo que lo emplea y el código y el esquema de datos que lo materializan. Richards y Ford (2020) destacan además su papel como criterio para decidir qué elementos pertenecen juntos al establecer la granularidad de los componentes de un sistema.

El propio enunciado proporciona el criterio de división: allí donde un mismo término sostiene significados incompatibles, pasa una frontera. Se proponen por tanto cuatro contextos delimitados.

Contexto	**Qué es la póliza**	**Reglas que hace cumplir**
Suscripción	Contrato con cláusulas, coberturas y vigencia	Tarificación, evaluación del riesgo, condiciones de emisión y renovación
Siniestros	Expediente contra el que se reclama	Cobertura del hecho declarado, estados de la reclamación, plazos
Facturación de primas	Fuente de ingresos recurrentes	Cuotas, recibos, vencimientos, tratamiento del impago
Peritaje en campo	Número de referencia de una inspección	Valoración de daños, evidencia fotográfica, cierre del informe

Cada contexto mantiene su propio modelo de póliza, con los atributos y el comportamiento que sus reglas requieren y sin ninguno de los que no requieren. Lo que comparten no es una entidad, sino una *identidad*: el identificador de la póliza, que permite correlacionar la información de los cuatro modelos sin que ninguno dependa de la estructura interna de los otros.

Cabe precisar que un contexto delimitado no es un módulo, una capa ni un microservicio. Es una frontera semántica y, aunque puede coincidir con una unidad de despliegue, se trata de decisiones de distinto nivel: la primera responde a la coherencia del modelo; la segunda, a requisitos de escalabilidad, autonomía de los equipos y operación. Identificar

ambas de forma automática es uno de los errores que Richards y Ford (2020) advierten al discutir la granularidad de los servicios.

Protección del modelo de Póliza

Delimitar los contextos no basta: hace falta un mecanismo que impida que los detalles de un contexto se filtren hacia el modelo de otro. Ese mecanismo descansa sobre dos piezas que operan de forma conjunta.

La primera es la ya mencionada ausencia de una entidad compartida. En el contexto de Suscripción, `Póliza` es una *raíz de agregado* (Evans, 2003): sus entidades y objetos de valor internos —cláusulas, coberturas, condiciones particulares— solo son alcanzables a través de la raíz, que es la responsable de hacer cumplir las invariantes del contrato. Ningún componente externo al agregado conserva una referencia directa a una cláusula suelta. Facturación no manipula esa estructura: mantiene su propia `CuentaDePrima`, que referencia la póliza únicamente por su identificador. Siniestros hace lo mismo con su `Expediente`, y Peritaje con su `OrdenDeInspección`.

La segunda es el Lenguaje Ubicuo de cada contexto, que actúa como criterio de admisión: un atributo solo entra en un modelo si el vocabulario de ese contexto lo reclama. Preguntas como «¿cuánto se debe de esta póliza?» o «¿qué fotografías tomó el ajustador?» no pertenecen al Lenguaje Ubicuo de Suscripción y, por tanto, no justifican atributos en su modelo.

El valor de estas dos restricciones se aprecia mejor por contraste. Supóngase el modelo unificado que se obtiene al ignorarlas, con una única clase que reúna cláusulas, coberturas, recibos pendientes, saldo deudor, siniestros abiertos, fotografías de los daños y la ubicación del ajustador. Las consecuencias son tres, en orden creciente de gravedad. Primera, un cambio en las reglas de facturación obliga a modificar y redesplegar el componente que sostiene la suscripción, pese a que las reglas actuariales no han cambiado. Segunda, las invariantes de las distintas áreas entran en conflicto dentro de una misma clase: para facturación una póliza con recibos impagados sigue existiendo y debe seguir generando cuotas, mientras que para suscripción puede haber quedado sin vigencia, y una sola clase no puede sostener ambas reglas a la vez sin recurrir a condicionales que dependen de quién la invoca. Tercera, y decisiva, la expresión «póliza activa» deja de tener un significado único en el código, que es exactamente la pérdida del Lenguaje Ubicuo que la delimitación de contextos venía a evitar.

Por la misma razón debe descartarse resolver la integración mediante una base de datos compartida sobre la que los cuatro contextos leen y escriben. Una tabla común de pólizas reintroduce el modelo unificado por la puerta del esquema relacional: las cuatro áreas vuelven a quedar acopladas a una misma estructura, y cualquier columna añadida para satisfacer a una de ellas pasa a formar parte del modelo de las demás. Evans (2003) contempla el patrón *Shared Kernel* para compartir de forma deliberada un subconjunto reducido del modelo, pero con un coste de coordinación permanente entre los equipos implicados; en un escenario de cuatro áreas con vocabularios divergentes, ese coste no se justifica.

Comunicación desacoplada entre los contextos

Los contextos permanecen separados pero no aislados: la emisión de una póliza debe producir cuotas, y la declaración de un siniestro debe originar una inspección. Evans (2003)

propone documentar esas relaciones de forma explícita en un *Context Map*, acompañado de un catálogo de patrones de integración. Tres de ellos resultan pertinentes aquí.

- **Capa de Anticorrupción** (*Anticorruption Layer*). Es el patrón indicado para consumir un modelo ajeno que no se controla, como un sistema de facturación de primas preexistente. La capa traduce el modelo externo al propio en la frontera, de modo que el vocabulario y las decisiones de diseño del sistema ajeno no penetren en el modelo de dominio.
- **Open Host Service** con **Published Language**. Suscripción atiende a varios consumidores, por lo que en lugar de exponer su modelo interno publica un contrato estable y versionado, expresado en un lenguaje de intercambio acordado. Así puede evolucionar su modelo interno sin romper a sus consumidores, siempre que respete el contrato publicado.
- **Customer/Supplier**. Describe la relación entre Siniestros y Peritaje en campo: el segundo provee al primero y ambos pertenecen a la misma organización, de modo que las necesidades del consumidor pueden negociarse y planificarse en lugar de simplemente padecerse.

Resta justificar por qué la comunicación debe ser predominantemente asíncrona, y el argumento no es de oportunidad tecnológica sino una consecuencia del modelo. La regla de consistencia de los agregados establece que una transacción modifica un solo agregado (Evans, 2003); en consecuencia, en el momento en que una operación de negocio atraviesa la frontera de un contexto, la consistencia transaccional deja de estar disponible por construcción. Lo que queda es la publicación de hechos del dominio —*póliza emitida*, *siniestro declarado*, *prima impagada*— mediante mensajería, y la reacción autónoma de cada contexto, que mantiene su propia proyección de aquello que necesita saber del resto.

El compromiso correspondiente debe declararse con honestidad. La integración asíncrona aporta autonomía operativa y tolerancia a la indisponibilidad temporal de un contexto, pero a cambio introduce consistencia eventual: existen ventanas durante las cuales Facturación aún no conoce una póliza ya emitida. Sostener ese diseño exige razonar sobre el orden y la posible duplicación de los mensajes, y aceptar que ciertas preguntas solo admiten respuesta aproximada hasta que el sistema converge; se trata de uno de los compromisos característicos de la integración distribuida que analizan Richards y Ford (2020).

Arquitectura orientada al dominio y Adaptador de Servicio

La solución anterior describe *dónde* están las fronteras, pero no *cómo* se sostienen dentro de cada contexto. Esa es la aportación de las arquitecturas orientadas al dominio. La Arquitectura Cebolla organiza el sistema en capas concéntricas con el Modelo de Dominio en el centro, de manera que todas las dependencias apuntan hacia adentro y las capas externas implementan interfaces definidas en las internas (Palermo, 2008). La Arquitectura Limpia formula el mismo compromiso como *Regla de Dependencia*: las dependencias del código fuente solo pueden apuntar hacia adentro, hacia las políticas de mayor nivel (Martin, 2017).

La relación entre ambos cuerpos de ideas es de complementariedad y opera en niveles distintos: DDD determina *qué* ocupa el centro —el modelo de dominio de ese contexto, con su Lenguaje Ubicuo— mientras que la arquitectura determina *cómo* se lo protege,

fijando la dirección de las dependencias. De ahí se sigue una decisión concreta de diseño: no existe una única cebolla para todo el sistema, sino que cada contexto delimitado recibe su propia instancia de la arquitectura, con su propio modelo de dominio en el centro y su propia infraestructura en el exterior.

Queda por ubicar la traducción entre contextos. La Guía de Arquitectura N-Capas orientada al Dominio sitúa en la capa de Infraestructura unos componentes denominados *Adaptadores de Servicio*, cuya responsabilidad es encapsular la invocación de servicios externos y traducir los datos que estos devuelven al modelo propio, de modo que ni el protocolo ni el vocabulario ajenos asciendan hacia las capas de dominio y aplicación (De la Torre Llorente et al., 2010).

Esto permite cerrar el razonamiento: el Adaptador de Servicio es el lugar donde se materializa la Capa de Anticorrupción. El núcleo de dominio declara la interfaz que necesita —por ejemplo, un `ServicioDeFacturación` expresado en los términos de su propio Lenguaje Ubicuo— y el adaptador, residente en la capa de infraestructura, la implementa contra el protocolo concreto del sistema de primas y efectúa la traducción entre ambos modelos. La interfaz pertenece a la capa interna y la implementación a la externa, con lo que la dependencia del código fuente apunta hacia adentro y la Regla de Dependencia queda satisfecha. Es la misma construcción que Cockburn (2005) describe como puertos y adaptadores, con el puerto en poder del interior.

Dos consecuencias prácticas se desprenden de esta disposición. La primera es la testabilidad: al depender el dominio de una abstracción propia y no del cliente concreto del sistema externo, su lógica puede ejercitarse con una implementación sustituta de la interfaz, sin invocar el servicio de primas ni la biblioteca de comunicación que lo consume. La segunda es la capacidad de evolución: sustituir el sistema de facturación, o su protocolo, afecta a un único adaptador en la infraestructura y deja intacto el modelo de dominio, que es la porción del sistema cuya estabilidad interesa preservar.

Síntesis

La solución propuesta se articula en tres niveles que responden a preguntas distintas y que deben sostenerse simultáneamente. En el nivel estratégico, DDD determina dónde situar las fronteras: cuatro contextos delimitados, deducidos de los significados incompatibles que el dominio asigna al término póliza, cada uno con su Lenguaje Ubicuo y su propio modelo. En el nivel estructural, las arquitecturas Cebolla y Limpia fijan la dirección de las dependencias dentro de cada contexto y mantienen el modelo de dominio libre de detalles de infraestructura. En el nivel de implementación, el Adaptador de Servicio concreta la traducción en la frontera y da forma a la Capa de Anticorrupción, de manera que el modelo protegido en los dos niveles anteriores sobreviva al contacto con sistemas que la organización no controla.

4. Pregunta 4: Evaluación de la Inversión de Dependencias en la práctica

Para juzgar la calidad de diseño de la clase `OrderProcessor` es necesario definir el marco normativo que rige la construcción de software orientado a objetos y la estructuración de arquitecturas limpias. A partir de él se responden, en orden, los tres incisos del enunciado: la violación de los principios (inciso a), el rediseño de la interacción (inciso b) y las pruebas

aisladas y la mantenibilidad (inciso c).

4.1. Marco normativo: DIP y Regla de Dependencia

Principio de Inversión de Dependencias (DIP)

Formulado por Robert C. Martin, el DIP es el último de los cinco principios SOLID (Martin, 2017). Establece que las estructuras de software más flexibles y mantenibles son aquellas donde las dependencias del código fuente se refieren exclusivamente a abstracciones y nunca a clases concretas (Martin, 2017). El principio se desglosa formalmente en dos reglas:

1. Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.

2. Las abstracciones no deben depender de los detalles; los detalles deben depender de las abstracciones.

La Regla de Dependencia (*The Dependency Rule*)

En la literatura de Clean Architecture, Robert C. Martin extiende el DIP al nivel de componentes organizados en capas concéntricas (Martin, 2017). Las dependencias del código fuente solo pueden apuntar hacia adentro, en dirección hacia las políticas de mayor nivel y el núcleo de dominio. Nada declarado en un círculo o capa interna (entidades, casos de uso, interfaces) debe conocer, mencionar o importar elemento alguno perteneciente a un círculo externo (Martin, 2017).

4.2. Inciso a): Violación de los principios arquitectónicos

En la implementación descrita en el ejercicio, `OrderProcessor` instancia y llama directamente a la clase `AddressValidatorService`. Esta instanciación es lo que constituye la dependencia directa y genera un acoplamiento rígido con la librería HTTP subyacente. A continuación se analiza esa violación frente a cada uno de los tres estilos.

Arquitectura Hexagonal (Puertos y Adaptadores)

Principios fundamentales. Propuesta por Alistair Cockburn (Cockburn, 2005), la Arquitectura Hexagonal nace para evitar que los detalles tecnológicos contaminen la lógica central de la aplicación. Sus pilares esenciales son:

Distinción dentro/fuera (*inside* vs. *outside*): Elimina la visión apilada tradicional («arriba y abajo») y establece que solo existe una frontera relevante: el interior del hexágono (la lógica de la aplicación) contra el exterior del hexágono (actores, interfaces de usuario, base de datos y servicios externos) (Cockburn, 2005).

Simetría de actores exteriores: Desde la perspectiva del núcleo de la aplicación, una llamada proveniente de una interfaz de usuario, una prueba unitaria automatizada en memoria o una petición recibida por una red HTTP son simétricas: eventos externos que intentan interactuar con el sistema (Cockburn, 2005).

Puertos (*ports*): Un puerto es una interfaz abstracta declarada y poseída por el núcleo. Define un contrato de comunicación agnóstico a la tecnología (Cockburn, 2005):

- **Puertos primarios o conductores (*driving*):** ubicados en la entrada, por donde los actores externos invocan las funcionalidades de la aplicación.
- **Puertos secundarios o conducidos (*driven*):** ubicados en la salida, por donde la aplicación consulta o envía mensajes hacia la infraestructura externa.

Adaptadores (*adapters*): Un adaptador es un componente de infraestructura concreto que rodea a un puerto, actuando como puente de traducción entre el mundo exterior y los contratos del núcleo (Cockburn, 2005).

Ejecución aislada (*headless mode*): El núcleo debe ser capaz de ejecutarse y probarse al 100 % en modo «desacoplado», usando *mocks* o adaptadores de prueba en lugar de componentes técnicos reales (Cockburn, 2005).

Violación en `OrderProcessor`.

- **Invocación directa sin puerto secundario:** `OrderProcessor` (el *inside* del hexágono) invoca directamente la clase técnica `AddressValidatorService` (*outside* del hexágono) sin interponer una interfaz o puerto secundario en la frontera del núcleo.
- **Infiltración de adaptadores de salida en el núcleo:** El principio de Cockburn exige que el código externo no invada el interior del hexágono (Cockburn, 2005). Al instanciar directamente el adaptador técnico de salida, las dependencias de la librería HTTP invaden la clase del negocio.
- **Ruptura del modo desacoplado:** Al estar soldado a la librería HTTP real, es imposible ejecutar `OrderProcessor` en un entorno de pruebas sin contar con conexión a red y un servidor HTTP activo, lo que destruye la testeabilidad de la arquitectura.

Arquitectura Cebolla (*Onion*)

Principios fundamentales. Propuesta por Jeffrey Palermo (Palermo, 2008), la Arquitectura Cebolla está diseñada para aplicaciones empresariales de larga vida y se estructura en capas concéntricas. Sus cuatro principios clave son (Palermo, 2008):

- **Construcción alrededor de un modelo de objetos independiente:** El centro absoluto de la cebolla es el modelo de dominio (*Domain Model*), que contiene el estado y el comportamiento esenciales del negocio y solo está acoplado a sí mismo.
- **Las capas internas definen las interfaces y las externas las implementan:** La capa central no guarda el código de acceso a datos ni clientes HTTP; únicamente contiene los contratos o interfaces. La implementación concreta de esas interfaces se empuja hacia los bordes exteriores.
- **Dirección de acoplamiento estrictamente hacia el centro:** Todo el código de las capas exteriores puede depender de las capas internas, pero el código de las capas internas jamás puede depender de capas más externas.

- **Compilación y ejecución independiente del núcleo:** Todo el *Application Core* debe poder compilarse y ejecutarse de manera completamente aislada de la infraestructura.

Violación en `OrderProcessor`.

- **Contaminación del centro por la infraestructura:** En lugar de empujar la tecnología HTTP hacia el borde exterior de la cebolla, la implementación de `OrderProcessor` trae la llamada HTTP al corazón de la aplicación.
- **Inversión negativa del acoplamiento:** El acoplamiento viaja desde el centro (*Application Core*) hacia afuera (infraestructura), violando directamente la norma de que «todo acoplamiento es hacia el centro» (Palermo, 2008).
- **Acoplamiento a tecnologías cambiantes:** Palermo enfatiza que la infraestructura es lo que cambia con mayor frecuencia en la industria (Palermo, 2008). Al tener a `OrderProcessor` acoplado a `AddressValidatorService` y a su librería HTTP, cualquier actualización o descontinuación de la librería obligará a modificar el código fuente de la lógica de procesamiento de pedidos.

Arquitectura Limpia (*Clean Architecture*)

Principios fundamentales. Robert C. Martin sintetiza las ideas de Hexagonal y Onion definiendo cuatro círculos concéntricos principales (Martin, 2017):

- **Entidades (*Entities*):** reglas de negocio críticas y globales.
- **Casos de uso (*Use Cases/Interactors*):** políticas específicas de la aplicación que coordinan el flujo de datos.
- **Adaptadores de interfaz (*Interface Adapters*):** presentadores, controladores y *gateways* que convierten datos.
- **Frameworks y Drivers:** herramientas externas, bases de datos, UI y clientes HTTP.

Sus principios rectores son (Martin, 2017):

- **Núcleo como la política de mayor nivel:** Las reglas de negocio ocupan la posición más privilegiada de la jerarquía y deben protegerse de cualquier cambio en las capas externas.
- **La infraestructura como un *plugin*:** Las bases de datos, la interfaz web o las librerías HTTP deben comportarse como meros complementos intercambiables que se conectan al sistema sin alterar el núcleo.
- **Cruce de fronteras mediante polimorfismo:** Para que el flujo de control ejecute código en la infraestructura sin violar la Regla de Dependencia, se utiliza el DIP: el caso de uso llama a un puerto o interfaz interno, y la infraestructura lo implementa.

Violación en `OrderProcessor`.

- **Violación de la dirección de la Regla de Dependencia:** `OrderProcessor` reside en la capa de casos de uso (política de alto nivel), mientras que `AddressValidatorService` reside junto a la librería HTTP en la capa de Frameworks y Drivers (mecanismos de bajo nivel). La flecha de dependencia apunta directamente hacia afuera, rompiendo el principio arquitectónico fundamental.
- **Destrucción del concepto de *plugin*:** Debido a que `OrderProcessor` utiliza la instanciación directa, el servicio de validación de direcciones no es un *plugin* reemplazable, sino una pieza «soldada» dentro del módulo central de la aplicación.

Dirección de las dependencias y por qué la implementación actual no la cumple

Las dependencias del código fuente deben apuntar estrictamente de afuera hacia adentro. El código de alto nivel (dominio y casos de uso) jamás debe mencionar, importar ni instanciar clases pertenecientes a módulos de bajo nivel (bases de datos, clientes HTTP, UI) (Martin, 2017). La implementación actual no lo cumple por dos razones:

- La flecha de dependencia apunta en **dirección contraria (de adentro hacia afuera)**.
- La clase de alto nivel `OrderProcessor` (núcleo de negocio) conoce, depende e instancia directamente a la clase técnica de bajo nivel `AddressValidatorService` (infraestructura), quedando atada a su librería HTTP subyacente.

4.3. Inciso b): Rediseño arquitectónico

Inversión de dependencias mediante una interfaz

Se aplica el Principio de Inversión de Dependencias (DIP) (Martin, 2017) junto con el patrón de interfaz separada. El Código 2 muestra la interfaz y el caso de uso, que residen en el núcleo, y el Código 3, el adaptador que reside en la infraestructura.

```
1  // Puerto / interfaz declarada en el centro (propiedad del negocio)
2  public interface IAddressValidator
3  {
4      bool ValidateAddress(Address address);
5  }
6
7  // Caso de uso / lógica de negocio que depende únicamente de la abstracción
8  public class OrderProcessor
9  {
10     private readonly IAddressValidator _addressValidator;
11
12     // Inyección de dependencias por constructor
13     public OrderProcessor(IAddressValidator addressValidator)
14     {
15         _addressValidator = addressValidator
16             ?? throw new ArgumentNullException(nameof(addressValidator));
17     }
18
19     public void ProcessOrder(Order order)
20     {
```

```
        // La regla de negocio invoca el contrato sin conocer detalles de red
        bool isValid = _addressValidator.ValidateAddress(order.ShippingAddress);

        if (!isValid)
        {
            throw new InvalidOperationException("La dirección de envío no es
    \ válida");
        }
        // lógica adicional para procesar el pedido...
    }
}
```

Código 2: Interfaz `IAddressValidator` y clase `OrderProcessor` en el núcleo

```
// Adaptador concreto en la infraestructura que implementa la interfaz del núcleo
public class AddressValidatorService : IAddressValidator
{
    private readonly HttpClientLibrary _httpClient;

    public AddressValidatorService(HttpClientLibrary httpClient)
    {
        _httpClient = httpClient;
    }

    public bool ValidateAddress(Address address)
    {
        // Detalle técnico: traducción de la llamada y comunicación HTTP
        var response = _httpClient.Post("/api/address/validate", address);
        return response.IsSuccess;
    }
}
```

Código 3: Adaptador `AddressValidatorService` en la infraestructura

Ubicación de los componentes por estilo

Arquitectura Hexagonal.

- **Núcleo de la aplicación (interior del hexágono / *inside*):**
  - `IAddressValidator`: reside dentro del hexágono actuando como un puerto secundario o conducido (*driven port*) (Cockburn, 2005). Define la conversación abstracta necesaria sobre la frontera del negocio.
  - `OrderProcessor`: reside dentro del hexágono como la lógica central de la aplicación que utiliza el puerto secundario.
- **Capa de infraestructura (exterior del hexágono / *outside*):**
  - `AddressValidatorService`: reside en el exterior del hexágono como un adaptador secundario o conducido (*driven adapter*) (Cockburn, 2005). Implementa el puerto e interactúa directamente con el dispositivo o servicio HTTP externo.

Arquitectura Cebolla.

- **Núcleo de la aplicación (*Application Core* / anillos internos):**
  - `IAddressValidator`: reside en las capas internas del núcleo de aplicación. Palermo establece expresamente que «las capas internas definen las interfaces» (Palermo, 2008).
  - `OrderProcessor`: reside en los servicios del núcleo de aplicación, acoplado únicamente a la interfaz y al modelo de objetos independiente.
- **Capa de infraestructura (anillo exterior):**
  - `AddressValidatorService`: reside en el borde más externo, en la infraestructura. Implementa la interfaz del núcleo e integra la librería HTTP concreta.
  - En tiempo de ejecución, un contenedor de Inversión de Control (IoC) resuelve la clase técnica de la infraestructura e inyecta su instancia en el constructor de `OrderProcessor`.

Arquitectura Limpia.

- **Núcleo de dominio y casos de uso (círculos concéntricos internos):**
  - `IAddressValidator`: reside en la capa de casos de uso actuando como una interfaz de límite de salida (Martin, 2017).
  - `OrderProcessor`: reside en la capa de casos de uso, coordinando el flujo de datos y aplicando las reglas de negocio.
- **Capa de infraestructura y *drivers* (círculos concéntricos externos):**
  - `AddressValidatorService`: reside en el círculo exterior de Adaptadores de Interfaz y Frameworks y Drivers. Implementa el puerto de salida del caso de uso mediante polimorfismo dinámico.

Justificación del diseño

La reestructuración realizada se justifica mediante la segunda regla del Principio de Inversión de Dependencias (DIP) (Martin, 2017), la cual establece que las abstracciones no deben depender de los detalles, sino que los detalles deben depender de las abstracciones.

- **Inversión de la flecha de compilación:** En la implementación original defectuosa, `OrderProcessor` (negocio) conocía y dependía de `AddressValidatorService` (detalle técnico). En el rediseño, la clase técnica `AddressValidatorService` debe importar e implementar la interfaz `IAddressValidator`, por lo que el detalle técnico de la infraestructura ahora depende explícitamente de la abstracción definida por el negocio.
- **Aislamiento de la volatilidad de los detalles:** Los clientes o librerías HTTP son detalles volátiles que cambian con frecuencia por actualizaciones tecnológicas. Al hacer que la infraestructura dependa de la interfaz abstracta `IAddressValidator`, los cambios técnicos en la llamada HTTP quedan totalmente contenidos en la capa externa y no provocan ningún impacto ni recompilación en el núcleo de la aplicación.

- **Inyección de dependencias por constructor (DI):** `OrderProcessor` ya no utiliza instanciación directa. Mediante la inyección de dependencias por constructor, el negocio declara qué abstracción necesita para operar, delegando la provisión de la implementación concreta a un contenedor IoC al inicio de la aplicación.

4.4. Inciso c): Pruebas aisladas y mantenibilidad

Impacto en las pruebas aisladas

El rediseño mediante la Inversión de Dependencias (DIP) y la Inyección de Dependencias por constructor (DI) resuelve de manera definitiva los problemas de testeabilidad de la clase `OrderProcessor`. En particular, elimina la dependencia de la red y de servidores externos:

- **Diseño original:** Para probar la lógica de `OrderProcessor`, el entorno de pruebas estaba obligado a realizar peticiones HTTP reales a un servidor externo. Si el servidor caía, la red fallaba o la API externa cambiaba, las pruebas unitarias fallaban aunque el código de negocio estuviera correcto.
- **Diseño rediseñado:** Al depender de la interfaz `IAddressValidator`, `OrderProcessor` se vuelve completamente independiente del mecanismo de validación real y de la red.

Uso de objetos simulados (*mocks* y *stubs*)

En el arnés de pruebas unitarias se crea un objeto *mock* que implementa la interfaz `IAddressValidator` y se inyecta directamente en el constructor de `OrderProcessor`. Esto permite simular escenarios de prueba en memoria sin realizar llamadas de red (Código 4).

```
[Fact]
public void ProcessOrder_DireccionInvalida()
{
    var mockValidator = new Mock<IAddressValidator>();

    mockValidator
        .Setup(v => v.ValidateAddress(It.IsAny<Address>()))
        .Returns(false);

    var processor = new OrderProcessor(mockValidator.Object);
    var order = new Order();

    Assert.Throws<InvalidOperationException>(() => processor.ProcessOrder(order));
}
```

Código 4: Prueba unitaria de `OrderProcessor` con un objeto simulado

Propiedades de calidad de las pruebas unitarias (FIRST)

- **Velocidad (*Fast*):** Las pruebas se ejecutan en milisegundos y en memoria, al no depender del tiempo de respuesta de E/S de la red.
- **Aislamiento (*Isolated*):** Se prueba exclusivamente la lógica de procesamiento del pedido, sin interferencia de componentes de infraestructura.

- **Repetibilidad (*Repeatable*):** Las pruebas producen exactamente el mismo resultado en cualquier entorno (máquina del desarrollador, servidor de integración continua CI/CD) sin requerir conexión a Internet.

Impacto en la mantenibilidad del software

La mantenibilidad mide la facilidad con la que el software puede modificarse para corregir errores, adaptarse a nuevos requisitos o evolucionar tecnológicamente. El rediseño optimiza este atributo de calidad a través de dos principios fundamentales de SOLID (Martin, 2017):

- **Principio de Responsabilidad Única (SRP):**
  - `OrderProcessor` adquiere una sola razón para cambiar: modificaciones en las reglas de negocio relativas al procesamiento de pedidos.
  - Si el servicio externo de validación de direcciones cambia sus *endpoints*, actualiza su formato JSON o migra de librería HTTP, la clase `OrderProcessor` no sufre ningún cambio ni requiere recompilación. El impacto del cambio queda contenido dentro de la clase `AddressValidatorService`, en la capa de infraestructura.
- **Principio Abierto/Cerrado (OCP):** El sistema queda abierto a la extensión pero cerrado a la modificación:
  - Si en el futuro la empresa decide cambiar el proveedor de validación de direcciones o utilizar una base de datos local para la validación, no se modifica una sola línea de código dentro de `OrderProcessor`.
  - Basta con crear una nueva clase en la infraestructura que implemente la interfaz `IAddressValidator` e inyectarla mediante el contenedor de Inversión de Control (IoC).

Reducción de la deuda técnica. Como enfatizan los autores de las arquitecturas Hexagonal, Cebolla y Limpia (Cockburn, 2005; Martin, 2017; Palermo, 2008), proteger el núcleo de la aplicación de la volatilidad de la infraestructura reduce los costos de mantenimiento y extiende el tiempo de vida útil del sistema.

5. Pregunta 5: El Kitty Problem y el Principio Abierto/Cerrado

5.1. Descripción del problema

El *Kitty Problem* parte de un sistema agregador de taxis dividido en servicios: `TaxiUI` (atiende al cliente), `TaxiFinder` (busca taxis disponibles), `TaxiSelector` (elige el más adecuado según coste, tiempo o lujo), `TaxiDispatcher` (ordena el servicio) y `TaxiSupervisor` (controla el viaje) (Martin, 2017). Los servicios se despliegan por separado y parecen desacoplados.

Un día se pide un requisito nuevo: la entrega de gatitos a domicilio. Hay conductores alérgicos a los gatos que deben excluirse, vehículos no aptos e instrucciones de recogida y devolución propias. El resultado es que *todos* los servicios deben modificarse, y esos

cambios deben coordinarse y desplegarse juntos. Los servicios estaban desacoplados en ejecución, pero no frente al cambio (Martin, 2017).

5.2. ¿Por qué la división por verbos provocó el problema?

Los servicios `TaxiFinder` o `TaxiSelector` representan *pasos del algoritmo* de reserva, no conceptos de negocio. Es una descomposición funcional: el concepto «tipo de viaje» queda repartido entre todos los servicios, de modo que cuando ese concepto cambia (aparecen los gatitos), el cambio se propaga a todos ellos. Richards y Ford (2020) lo describen como una partición técnica frente a una partición por dominio: en la primera, un cambio de dominio atraviesa todas las particiones; en la segunda, queda confinado a una sola. Además, todos los servicios comparten la misma representación del viaje, por lo que siguen acoplados a través de los datos.

5.3. Diseño basado en DDD y Onion/Clean

DDD propone dividir el sistema según los conceptos del dominio y sus ejes de cambio. Aquí la observación clave es que la entrega de gatitos *es un tipo de viaje*: comparte el flujo general (buscar, seleccionar, despachar, supervisar) pero varía en reglas concretas. Por tanto se separan:

- `Rides`: núcleo con la lógica estable del viaje y los puntos de extensión (clases abstractas e interfaces como `Ride` o `DriverEligibility`).
- `Kittens`: plugin que implementa esas abstracciones con las reglas propias de los gatitos (p. ej. excluir conductores alérgicos).

Siguiendo la Regla de Dependencia (Martin, 2017) y la Arquitectura Onion (Palermo, 2008), `Rides` ocupa el centro y `Kittens` depende de él, nunca al revés (Figura 1). Cada servicio puede mantener su despliegue, pero internamente opera sobre las abstracciones de `Rides` mediante polimorfismo (*Template Method* o *Strategy*), sin conocer el tipo concreto de viaje (Martin, 2017). Así, añadir gatitos solo afecta a un componente: `Kittens`.

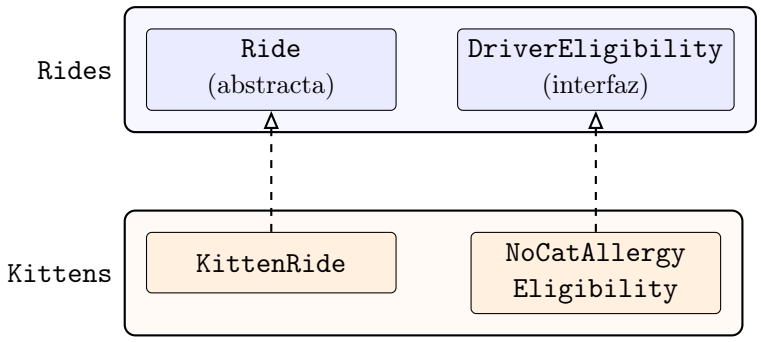


Figura 1: El plugin `Kittens` implementa las abstracciones del núcleo `Rides`. Las dependencias apuntan hacia el núcleo.

5.4. Principio de Abierto/Cerrado y papel de los Adaptadores

El OCP establece que un artefacto debe estar abierto a la extensión y cerrado a la modificación (Martin, 2017). En el diseño original, añadir gatitos exigía modificar cinco servicios. En el diseño propuesto, el núcleo `Rides` y los servicios existentes no se tocan

(cerrados a la modificación) y el nuevo comportamiento se añade creando el componente `Kittens` (abierto a la extensión).

Los Adaptadores hacen posible esta extensión. En términos de la Arquitectura Hexagonal (Cockburn, 2005), las abstracciones de `Rides` son *puertos* y `Kittens` es un *adaptador* que traduce las reglas de los gatitos al contrato que entiende el núcleo, que nunca conoce el concepto «gatito». Si los gatitos provinieran de un sistema externo, un adaptador de servicio encapsularía esa integración tras un puerto del dominio (De la Torre Llorente et al., 2010). Finalmente, el componente *Main* conecta el plugin mediante factorías o inyección de dependencias (Martin, 2017), por lo que incorporar un nuevo tipo de viaje se reduce a registrar un nuevo adaptador.

6. Pregunta 6: N-Capas frente a requisitos de atributos de calidad

Análisis de la afirmación del estudiante: El estudiante propone que el uso de capas abiertas (open layers) y el "fast-lane reader"mitigan los problemas de rendimiento de la Arquitectura N-Capas. Si bien estas técnicas mejoran el rendimiento puntual de las lecturas al evitar atravesar todas las capas, **no resuelven el problema de fondo** de los tres atributos de calidad mencionados (Richards, 2015; Richards & Ford, 2020):

- **Despliegue:** Sigue siendo monolítico. Cualquier cambio, por pequeño que sea, obliga a redesplegar toda la aplicación.
- **Escalabilidad:** Sigue siendo limitada. No se pueden escalar capas o módulos de forma independiente; hay que escalar el sistema completo.
- **Agilidad:** Sigue siendo baja debido al alto acoplamiento entre las capas, lo que dificulta realizar cambios aislados sin afectar el resto del sistema.

Impacto en el Aislamiento y la Mantenibilidad: La introducción de capas abiertas rompe el principio de aislamiento (Richards, 2015). Al permitir que una capa salte a otra inferior, se genera un alto acoplamiento. A largo plazo, esto destruye la mantenibilidad: los cambios en capas inferiores pueden romper múltiples capas superiores.

El dilema del anti-patrón "Sinkholez la regla 80/20: El "sinkhole.ºcurre cuando las peticiones pasan por las capas sin procesamiento real (Richards, 2015). Según Richards (2015), es normal y tolerable que alrededor del **20 %** de las peticiones sean sinkholes. El problema real (el anti-patrón) surge cuando esta proporción es del **80 %** o más, lo que indica que la arquitectura en capas es inadecuada para ese sistema. *La relación entre capas abiertas y sinkhole es inversa a lo que se suele pensar:* las capas abiertas se introducen como una **respuesta** para evitar el sinkhole y mejorar el rendimiento (Richards, 2015). Sin embargo, hacer esto compra rendimiento a costa de perder el aislamiento y generar acoplamiento, un trade-off que solo se justifica de forma muy controlada.

Idoneidad de Arquitecturas Hexagonal y Limpia: Para sistemas con altos requisitos de escalabilidad y agilidad, estas arquitecturas son superiores. A diferencia de N-Capas clásica (donde la capa de negocio depende de la de persistencia) (Richards, 2015), Hexagonal (Cockburn, 2005) y Limpia (Martin, 2017) aplican el principio de Inversión de Dependencias (la 'D' de SOLID) (Martin, 2017) y los conceptos de DDD (como Bounded Contexts) (Evans, 2003). Esto desacopla el dominio de la infraestructura. No son escalables "por sí mismas"mágicamente, pero al usar puertos y adaptadores (Cockburn,

2005), **facilitan enormemente** la separación de módulos y la extracción de servicios independientes, permitiendo una verdadera agilidad y escalabilidad.

Referencias

Cockburn, A. (2005). *Hexagonal Architecture (Ports and Adapters)*. https://alistair.cockburn.us/hexagonal-architecture/

De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. Á., & Calvarro Nelson, J. (2010). *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0* [Borrador]. Microsoft Ibérica.

Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Palermo, J. (2008). *The Onion Architecture*. https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/

Richards, M. (2015). *Software Architecture Patterns*. O'Reilly Media.

Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.

Declaración de Uso de la Inteligencia Artificial

Para la elaboración de este seminario, el equipo utilizó dos herramientas de inteligencia artificial. Por un lado, Gemini (versión Gemini 3.5), con el propósito de traducir al español fragmentos de las obras de la bibliografía y de obtener explicaciones introductorias de algunos estilos arquitectónicos, como la arquitectura hexagonal. Por otro lado, Claude (versión Claude Sonnet 5.5), con el propósito de generar ejemplos de código, ayudar a formalizar la redacción de las respuestas, dar el formato correcto a los archivos `.tex` y elaborar los esquemas. Kamila Reinoso Asín usó Gemini y el resto del equipo usó Claude. Todo el contenido generado fue revisado, verificado y adaptado por el equipo, quien asume la responsabilidad final del documento.